Progress report

License plate detection and recognition on CCPD2019

Status as of 27 October 2026 (progress-check milestone)

Code: `https://github.com/S1norin/inno-detecting-license-plates`

Contributors

Name	Email
Mikhail Krylov	m.krylov@innopolis.university
Maximilian Mifsud Bonici	m.mifsudbonici@innopolis.university
Andrei Polevoi	a.polevoi@innopolis.university
Evgenii Khovalyg	e.khovalyg@innopolis.university

1 Completed experiments

- **Dataset.** A 5,000-image CCPD2019 subset was prepared into 15,000 images with scale variants (1.0, 0.75, 0.5). Splits are assigned before resizing, so all variants of one photo stay in one split (3,500 / 700 / 800 original-scale train / validation / test).
- **E1 detection baseline.** The team's fine-tuned YOLO11s detector is used frozen, evaluated through the shared harness (CPU, single-threaded, identical inference settings for every model).
- **E2 recognition baseline.** Pretrained EasyOCR, frozen, under both protocols (`ocr_only` on ground-truth crops, `end_to_end` on detector boxes).
- **E2b plate-specific CRNN.** A 3-million-parameter CRNN + BiLSTM + CTC recognizer trained on the 3,500 original-scale training crops (50 epochs, Adam lr 0.001, batch 32, seed 42, CPU, single-threaded; selected epoch 39, validation character accuracy 0.7564).
- **E3 preprocessing ablation.** `none`, `gray_contrast`, and `canny`, each at its own validation-selected confidence threshold.
- **E4/E5 size and tilt sensitivity.** Per-bucket (small / medium / large) and per-tilt-bin (0–9, 10–19, 20+ degrees) metrics for detection and recognition.
- **E6 confidence threshold.** Per-character confidence with a validation-selected abstention threshold; test is evaluated exactly once per protocol with the saved policy.
- **E1 fine-tuning (new).** YOLO11s fine-tuned on the 3,500-image original-scale train split (640 px, batch 16, seed 42, deterministic, RTX 3090 Ti, ∼24 min). Early stopping ended the run after 24 of 100 epochs; best epoch 4, validation mAP@0.5 0.995.

- **End-to-end re-run (new).** The CRNN end-to-end protocol was re-run with the winning detector's predictions: validation ablation in all three preprocessing modes, mode and threshold selected from validation only, then a single test run.

2 Preliminary comparison

2.1 Detection: frozen vs fine-tuned YOLO11s

Test split, identical inference settings, shared harness:

Model	mAP@0.5	Precision	Recall	F1
Frozen YOLO11s	0.9073	0.9408	0.8938	0.9167
Fine-tuned YOLO11s	0.9949	0.9466	0.9975	0.9714

Table 1: Detection on the test split (800 images).

Per scope, test mAP@0.5 (frozen → fine-tuned): small 0.8453 → 0.9950, medium 0.9505 → 0.9950, large 0.9371 → 0.9948; tilt 0–9 0.9593 → 0.9947, 10–19 0.9018 → 0.9863, 20+ 0.8954 → 0.9950. The gain is dominated by recall (0.8938 → 0.9975); validation tells the same story (0.9245 → 0.9949).

2.2 Recognition: engines and detectors

Test split, 800 plates, both engines through the same harness. The end-to-end rows pair the CRNN with each detector's predicted boxes:

Protocol	Engine	Coverage	Char. acc.	Exact	Precision	Recall	F1
ocr_only	EasyOCR	0.7300	0.3474	0 / 800	0.0377	0.0924	0.0535
ocr_only	CRNN	1.0000	0.7538	269 / 800	0.5500	1.0000	0.7097
end_to_end	EasyOCR	0.7025	0.3240	0 / 800	0.0267	0.0593	0.0368
end_to_end	CRNN + frozen YOLO11s	0.9762	0.6561	133 / 800	0.3624	0.9371	0.5226
end_to_end	CRNN + fine-tuned YOLO11s	1.0000	0.7661	296 / 800	0.5837	1.0000	0.7372

Table 2: Recognition on the test split.

Preprocessing ablation for the current pipeline (CRNN + fine-tuned detector, validation split, each mode at its own selected threshold):

Mode	Coverage	Char. accuracy	Exact plates	Plate F1
none	0.9971	0.7534	237 / 700	0.7121
gray_contrast	1.0000	0.7500	222 / 700	0.7144
canny	0.9957	0.4539	5 / 700	0.0900

Table 3: End-to-end validation ablation; the shared rule selects `none`, tau 0.50.

3 Observations

- Fine-tuning the detector is the largest single gain of the project so far: test mAP@0.5 0.9073 → 0.9949, driven by recall (0.8938 → 0.9975). Small plates benefit most (0.8453 → 0.9950).
- The detector gain flows through the whole pipeline: end-to-end plate F1 rises 0.5226 → 0.7372, exact plates 133 → 296, and coverage reaches 1.0000 (no test abstentions).
- With the better detector, end-to-end character accuracy (0.7661) now slightly *exceeds* the ground-truth-crop number (0.7538): detector boxes no longer hurt recognition, unlike with the frozen detector (0.6561 vs 0.7538).
- The fine-tuned model converged quickly: the best epoch was 4 of 24 (early stop with patience 20), so the 100-epoch budget was far more than enough.
- Canny preprocessing collapses recognition (0.4539 vs 0.7534 character accuracy) and did so consistently for both detectors; the selection rule correctly avoids it. `gray_contrast` is a close second to `none` (validation F1 0.7144 vs 0.7121).
- Large plates remain the weakest recognition bucket (test plate F1 0.4073 vs 0.8750 for small) because they are downscaled into the fixed 48-px-tall input; the 10–19 degree tilt bin is the weakest (0.2500, 63 samples).
- Ultralytics `optimizer=auto` overrode the configured `lr0: 0.01` with AdamW lr 0.002; the actual optimizer state is recorded in the training record, which is the source of truth.
- The fine-tuned detector emitted one sub-pixel zero-area box on the test split; such boxes can never be true positives and are now filtered before metrics, with a regression test.
- Practical difficulties: training needed the machine's GPU freed, and the CRNN checkpoint (a gitignored artifact) had to be re-trained locally (∼70 min on CPU) before the end-to-end re-run; sharing it with the team is tracked as an open issue.

4 Remaining work

- **Large-plate recognition.** Taller CRNN input (e.g. 96 px) to reduce downscaling of large plates, re-trained and re-evaluated through the same harness.
- **Failure-case analysis.** At least three annotated failure examples for the results checkpoint.
- **Runtime and memory numbers.** Latency and peak memory for the full detection + recognition pipeline.
- **Reproducibility check.** Clean-clone run: regenerate the dataset, fetch pinned weights, and reproduce the reported metrics.
- **Checkpoint sharing.** Distribute the CRNN checkpoint (open issue) so teammates do not need to re-train it.
- **Results checkpoint, 3 November.** All experiments completed, final metrics, tables/figures, failure examples.

- **Final submission (10 November).** Two-page technical summary, organized and reproducible code with README and environment details, final slides, prepared demo, member contributions.

References

- Detection fine-tuning results: `models/yolo11_finetune/REPORT.md`
- CRNN recognizer: `models/crnn/REPORT.md`
- Recognition baseline: `baseline/ocr/README.md`
- Detection baseline: `baseline/detection/README.md`
- Repository: `https://github.com/S1norin/inno-detecting-license-plates`